# 24、Mcu休眠和唤醒测试

卖家'漩涡遨游aa' 专属资料，非在漩涡遨游aa 购买资料，不提供答疑(一次购买仅提供有限次数10次答疑)

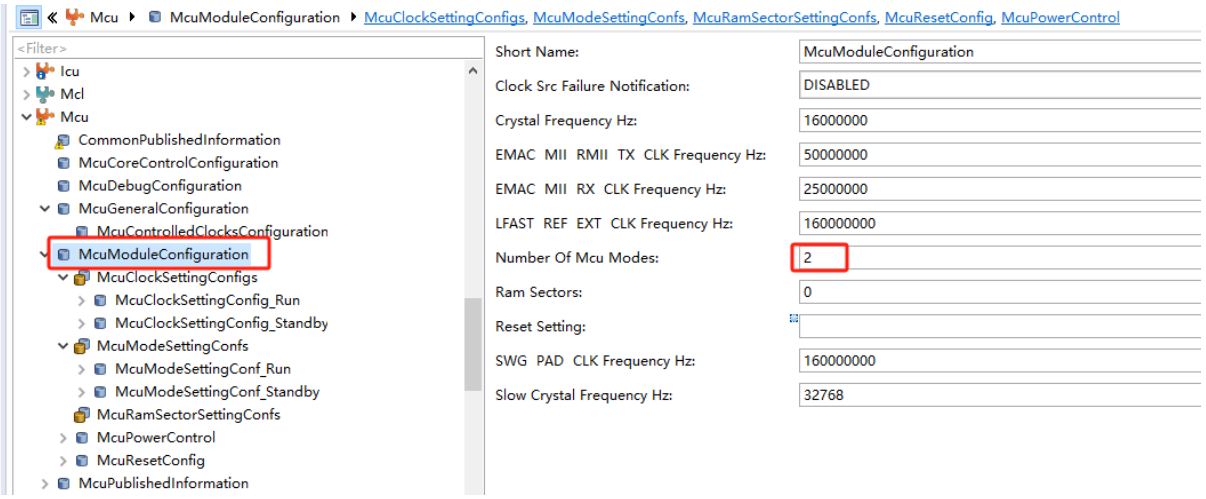
https://blog.csdn.net/HeFlyYoung/article/details/123681206

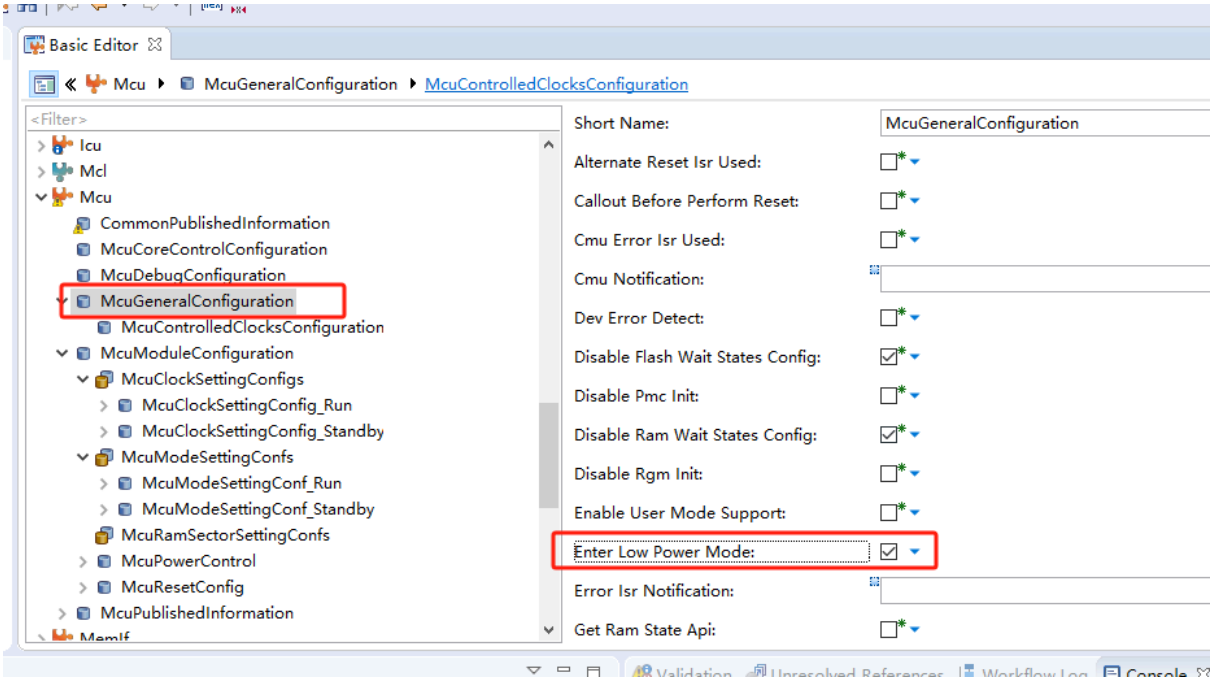
https://blog.csdn.net/weixin_43580890/article/details/139650446

https://blog.csdn.net/Abler_xaing/article/details/134771439

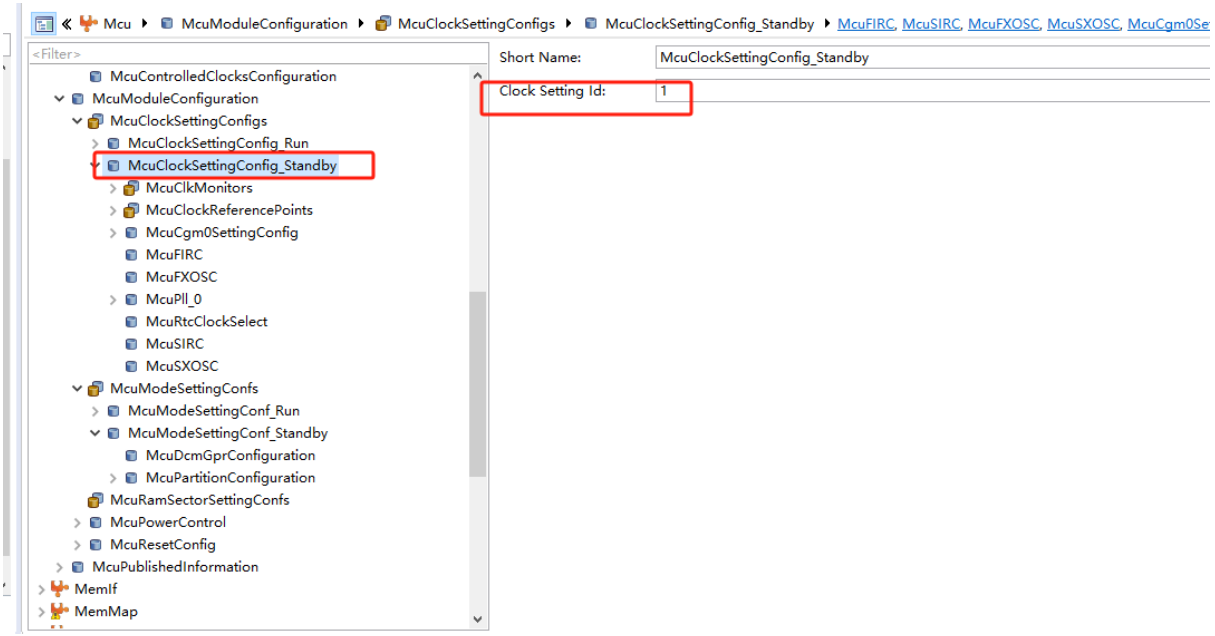
https://blog.csdn.net/studyingdda/article/details/127246248

## 一、添加Standby的mode和clock



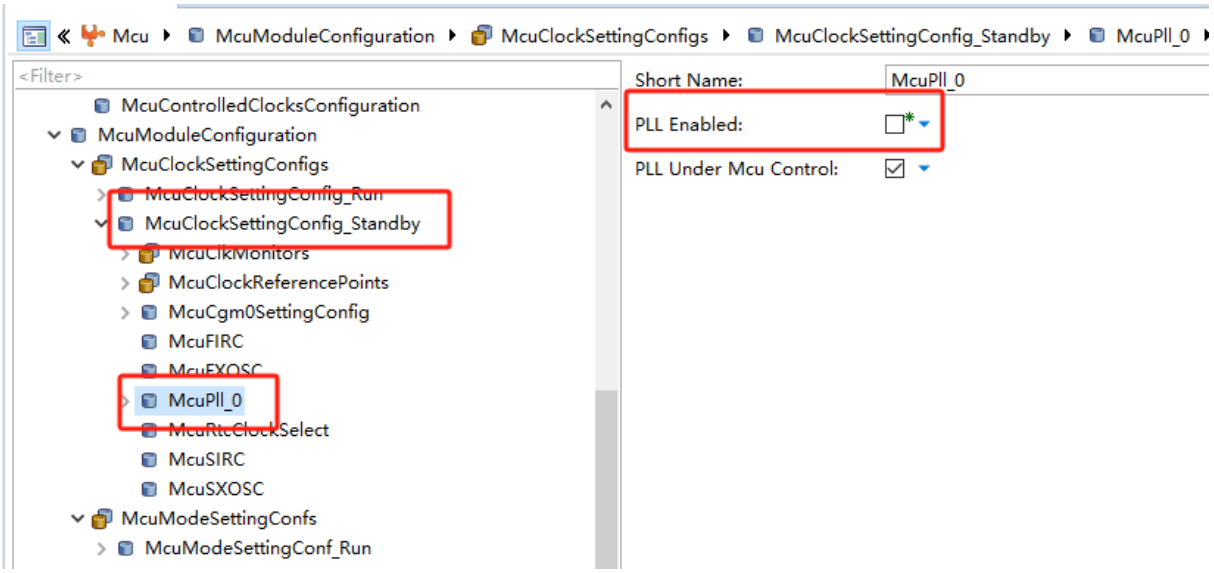


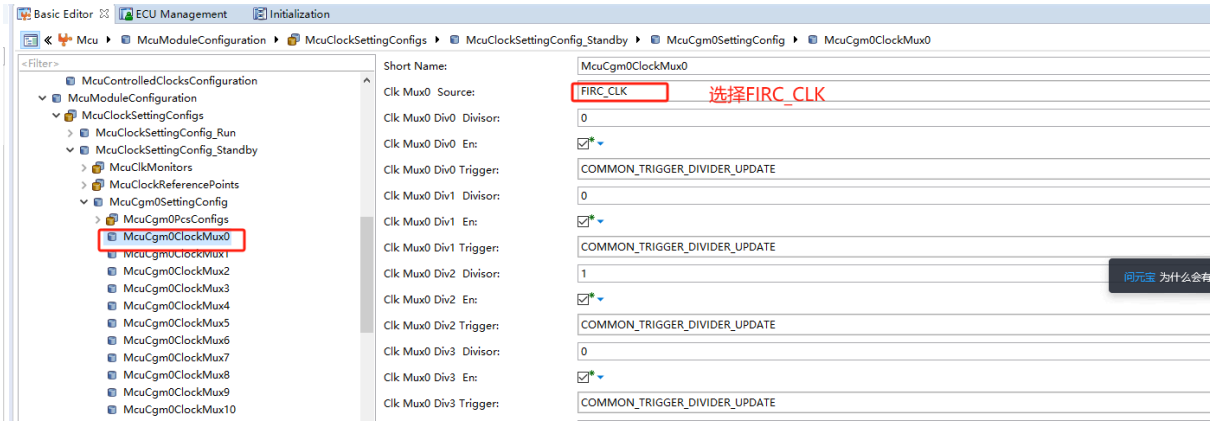

我们把原来的McuClockSettingConfig_1 改为McuClockSettingConfig_Standby



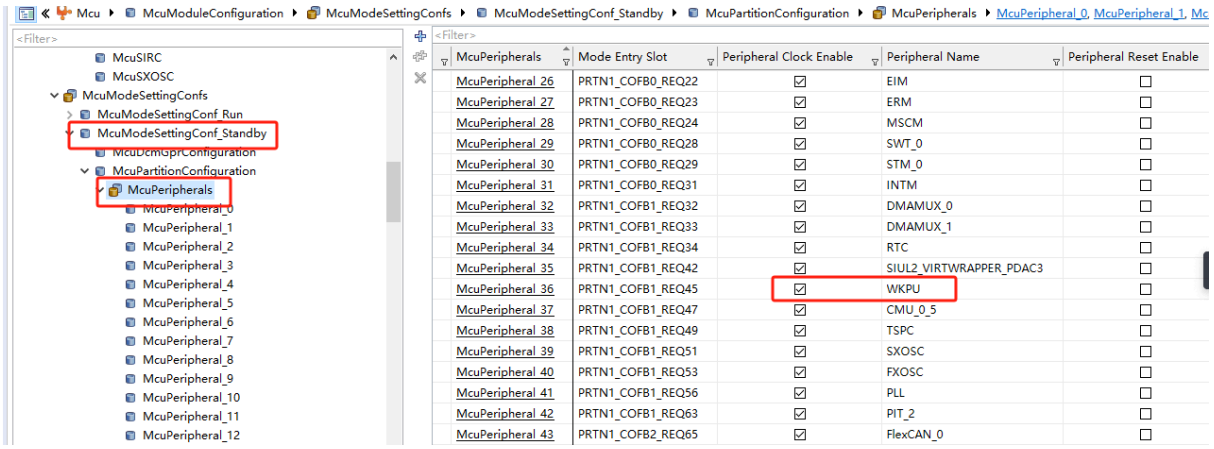

关键配置

关闭PLL

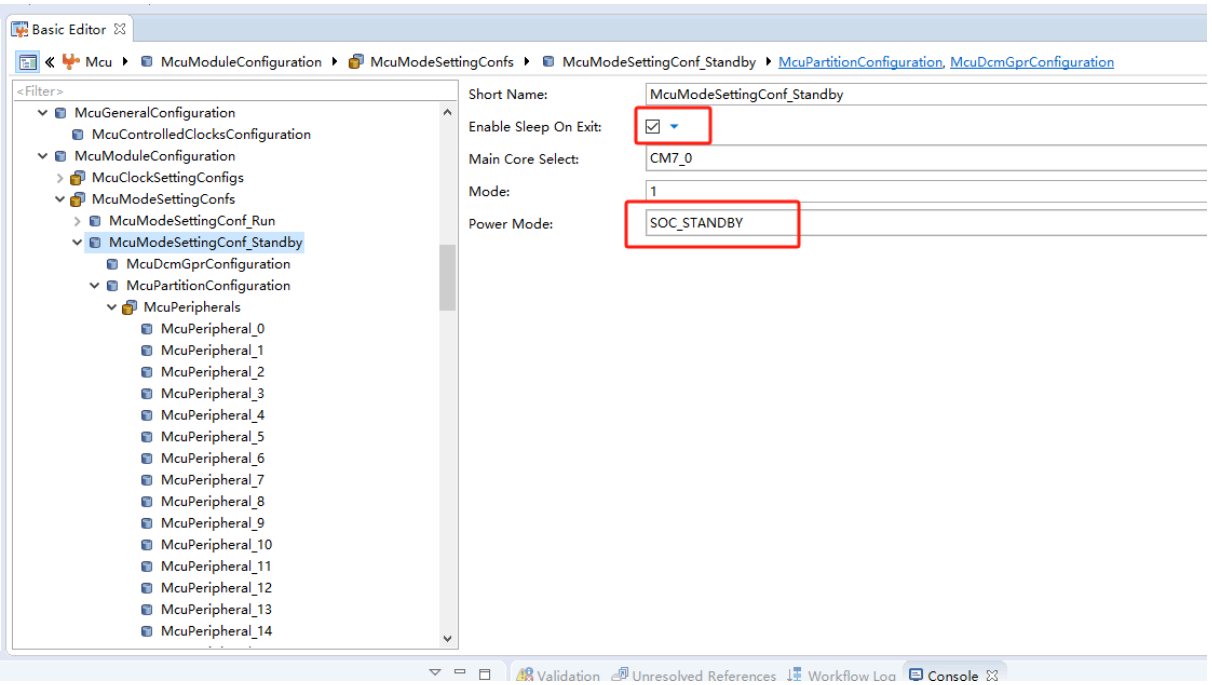



外设时钟可以根据需求勾选，这里因为要wakeup 唤醒，McuModeSettingConf_Standby的外设时钟必须勾选WKPU, 使用上一节的WKPU0 (PTA2) 下降沿唤醒



选择SOC_STANDBY



1.从图可以看出，RUN模式支持所有的外设，但Standy模式下仅深色支持部分，像锁相环PLL是不支持的，所以将FIRC作为Standy的时钟。S32K3只有两种工作模式，Run和 Standy。这部分主要去看手册的电源管理模块。

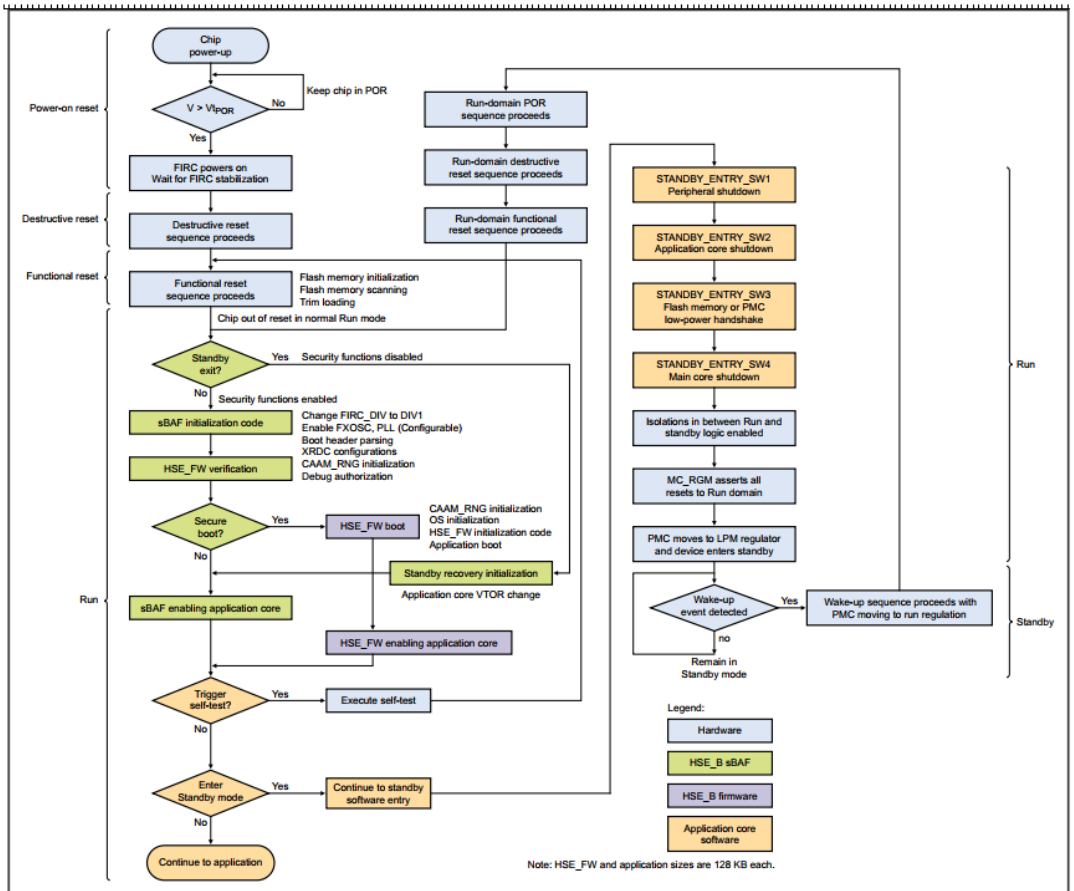
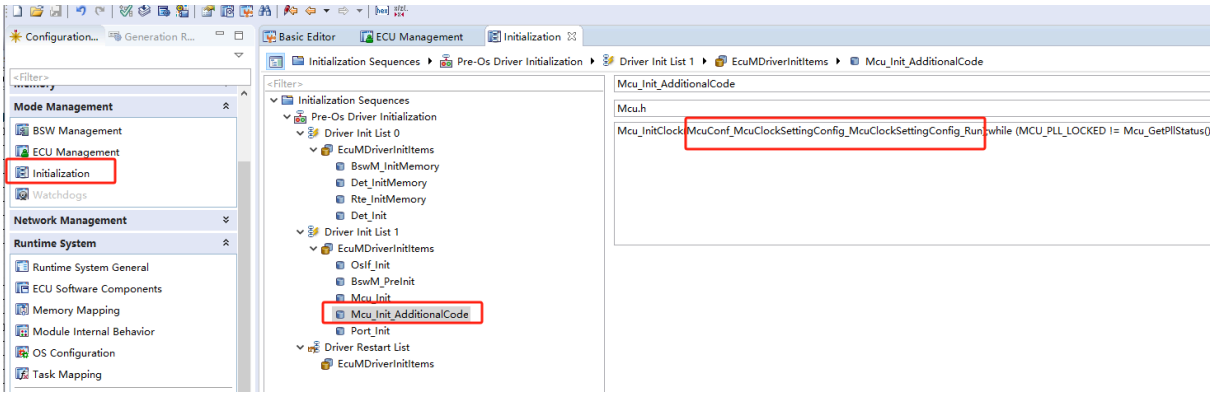


Figure 162. Power-up sequence

# 二、修改EcuM 中mcu的添加代码

因为我们把mode_0 和 clock_0 改为了 _standby，所以EcuM中mcu模块的额外代码也要改下



```
lockSettingConfig_Run);while (MCU_PLL_LOCKED != Mcu_GetPllStatus());Mcu_DistributePllClock();Mcu_SetMode(McuConf_McuModeSettingConf_McuModeSettingConf_Run);
```

编译所有模块

# 三、额外添加两路WKPU

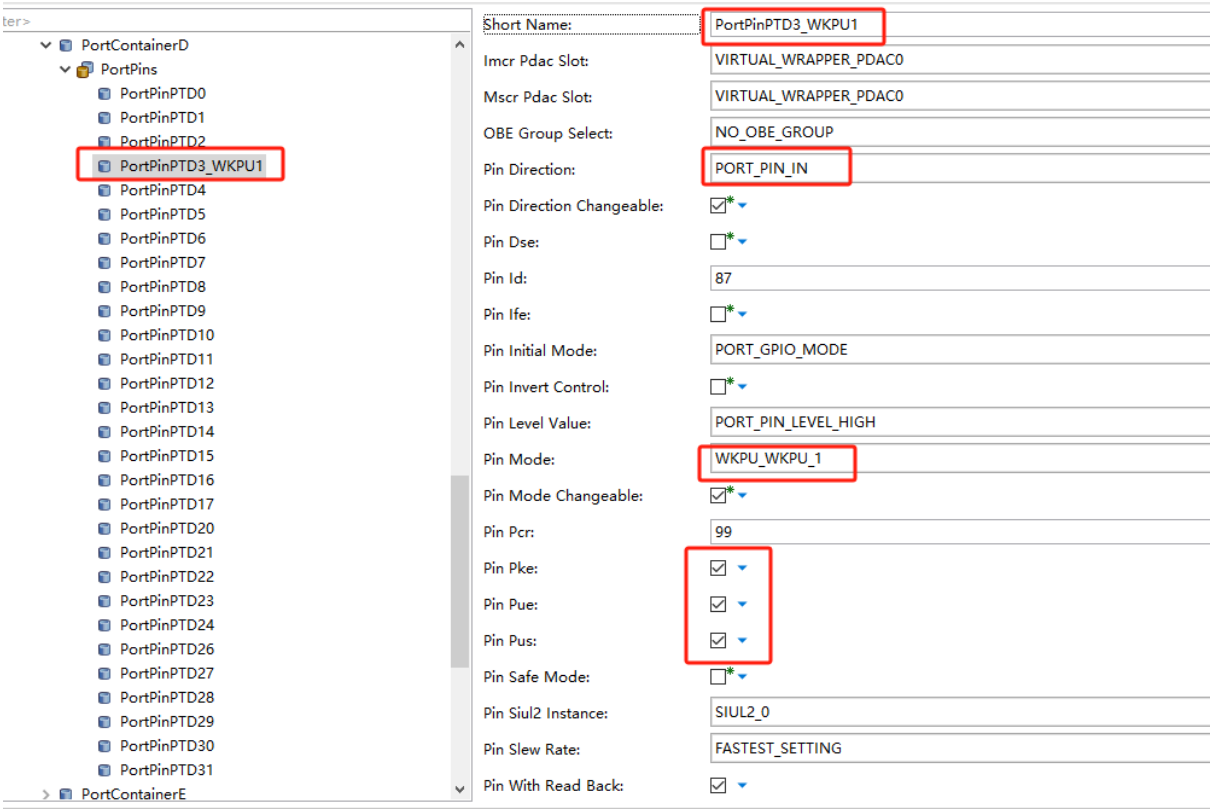
在 《Icu模块的Wkpu中断边沿》中，我们添加了PTA2的WKPU0，用作边沿中断回调测试的

现在我们多添加两路

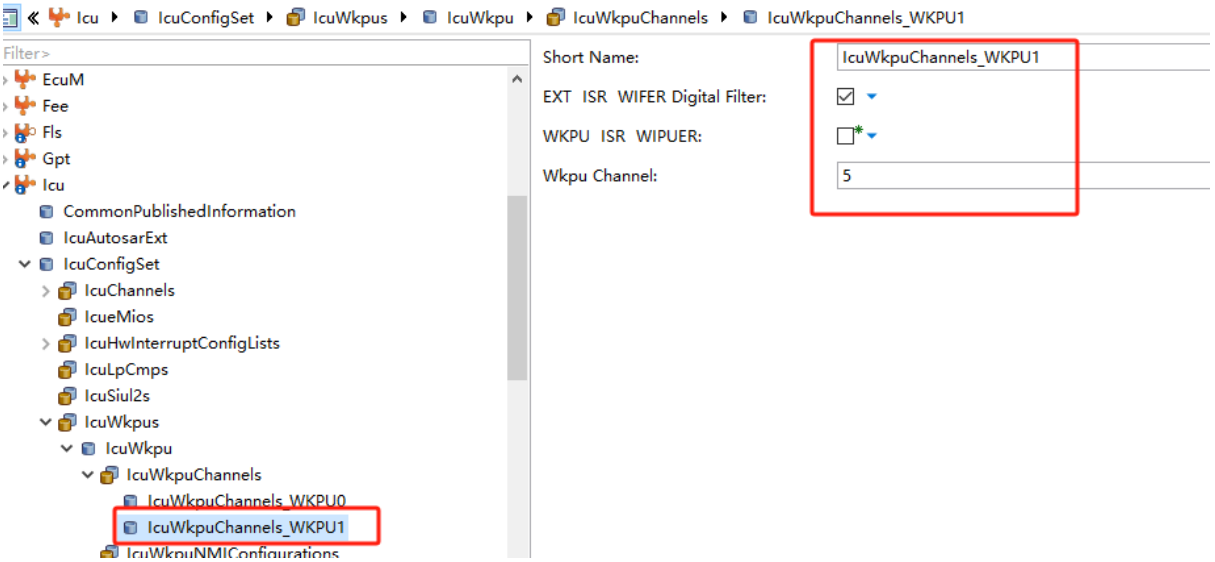
## 3.1、PTD3 WKPU1

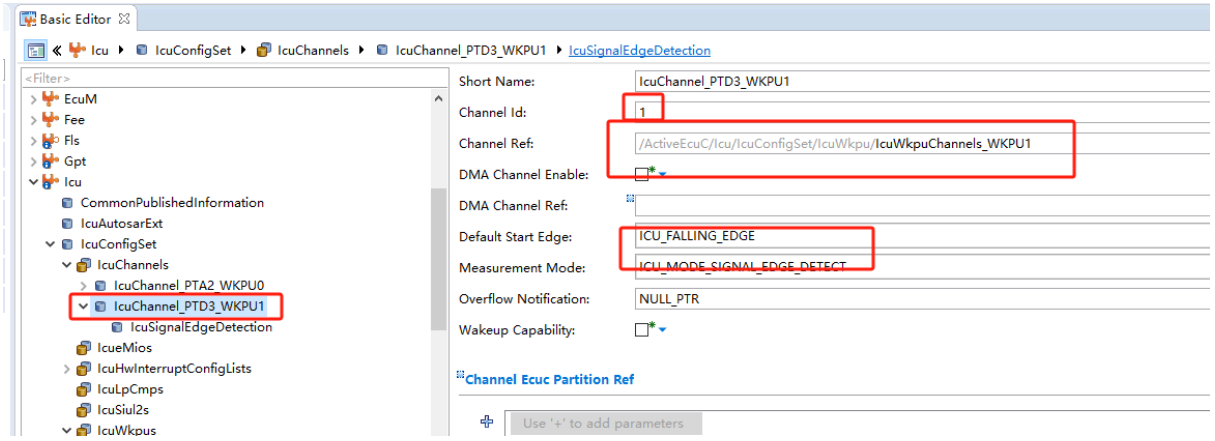
同理也是默认配置有上拉电阻，通过下降沿唤醒方式

port



icu




我们只用WKPU1 用来做休眠后的唤醒，所以我们不用配置他的其他ICU 功能呢，后面在休眠前调用 Icu_EnableEdgeDetection(IcuConf_IcuChannel_IcuChannel_PTD3_WKPU1)；来使能，既可以在休眠后唤醒mcu

## 3.2、PTC9_CAN1_RX WKPU10

https://blog.csdn.net/HeFlyYoung/article/details/123681206

说明：PTC9 配置为输入时WKPU10就是默认的路由功能，所以不用选WKPU10, 同理PTD3也是

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 6 | | SIUL_IMCR872 | 0000_0010 | LPUART0_CTS | LPUART0 | Clear To Send (bar) | I | |
| 7 | PTC9 | SIUL_MSCR73 | 0000_0000 | GPIO[73] | SIUL | | I/O | GPIO |
| 8 | | | 0000_0001 | LPI2C0_SDA | LPI2C0 | LPI2C Data I/O | O | |
| 9 | | | 0000_0010 | LPUART1_TX | LPUART1 | Transmit | O | |
| 0 | | | 0000_0011 | LPUART0_RTS | LPUART0 | Request To Send | O | |
| 1 | | | 0000_0100 | eMIOS_1_CH[8]_X | eMIOS_1 | eMIOS Channel | O | |
| 2 | | | 0000_0101 | LCU1_OUT6 | LCU1 | LCU Output | O | |
| 3 | | | 0000_0110 | LPSPI0_SIN | LPSPI0 | LPSPI Serial Data Input | O | |
| 4 | | | 0000_0111 | FXIO_D13 | FXIO | FlexIO Bi-directional Shift/timer I/O | O | |
| 5 | | - | - | ADC0_S13 | ADC0 | ADC Standard Input | I | |
| 6 | | - | - | WKPU[10] | WKPU | Wakeup Input | I | |
| 7 | | SIUL_IMCR513 | 0000_0001 | CAN1_RX | CAN1 | CAN Rx channel | I | |
| 8 | | SIUL_IMCR545 | 0000_0010 | EIRQ[17] | SIUL | External Interrupt | I | |
| 9 | | SIUL_IMCR600 | 0000_0001 | eMIOS_1_CH[8]_X | eMIOS_1 | eMIOS Channel | I | |
| 0 | | SIUL_IMCR677 | 0000_0011 | FXIO_D13 | FXIO | FlexIO Bi-directional Shift/timer I/O | I | |
| 1 | | SIUL_IMCR726 | 0000_0001 | LPI2C0_SDA | LPI2C0 | LPI2C Data I/O | I | |
| 2 | | SIUL_IMCR742 | 0000_0010 | LPSPI0_SIN | LPSPI0 | LPSPI Serial Data Input | I | |
| 3 | | SIUL_IMCR876 | 0000_0010 | LPUART1_TX | LPUART1 | Transmit | I | |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 06 | | SIUL_IMCR878 | 0000_0010 | LPUART3_TX | LPUART3 | Transmit | I | |
| 07 | PTD3 | SIUL_MSCR99 | 0000_0000 | GPIO[99] | SIUL | | I/O | GPIO |
| 08 | | | 0000_0010 | eMIOS_1_CH[22]_X | eMIOS_1 | eMIOS Channel | O | |
| 09 | | | 0000_0011 | LPSPI1_PCS0 | LPSPI1 | Peripheral Chip Select 0 | O | |
| 10 | | | 0000_0100 | FXIO_D5 | FXIO | FlexIO Bi-directional Shift/timer I/O | O | |
| 11 | | | 0000_0101 | FXIO_D7 | FXIO | FlexIO Bi-directional Shift/timer I/O | O | |
| 12 | | | 0000_0110 | LCU0_OUT0 | LCU0 | LCU Output | O | |
| 13 | | - | - | NMI_b | SYSTEM | Non Maskable Interrupt | I | |
| 14 | | - | - | WKPU[1] | WKPU | Wakeup Input | I | |
| 15 | | - | - | ADC0_S10 | ADC0 | ADC Standard Input | I | |
| 16 | | SIUL_IMCR539 | 0000_0011 | EIRQ[11] | SIUL | External Interrupt | I | |
| 17 | | SIUL_IMCR614 | 0000_0100 | eMIOS_1_CH[22]_X | eMIOS_1 | eMIOS Channel | I | |
| 18 | | SIUL_IMCR669 | 0000_0010 | FXIO_D5 | FXIO | FlexIO Bi-directional Shift/timer I/O | I | |
| 19 | | SIUL_IMCR671 | 0000_0011 | FXIO_D7 | FXIO | FlexIO Bi-directional Shift/timer I/O | I | |
| 20 | | SIUL_IMCR702 | 0000_0011 | LPUART3_RX | LPUART3 | Receive | I | |
| 21 | | SIUL_IMCR744 | 0000_0001 | LPSPI1_PCS0 | LPSPI1 | Peripheral Chip Select 0 | I | |
| 22 | | SIUL_IMCR860 | 0000_0001 | TRGMUX_IN4 | TRGMUX | Trigger Mux Input | I | |
| 23 | PTD4 | SIUL_MSCR100 | 0000_0000 | GPIO[100] | SIUL | | I/O | GPIO |

# 附：

**为什么不需要配置：**

因为Wkpu等一些特殊功能时默认路由的，在硬件上是直接选通的。

**例子：**

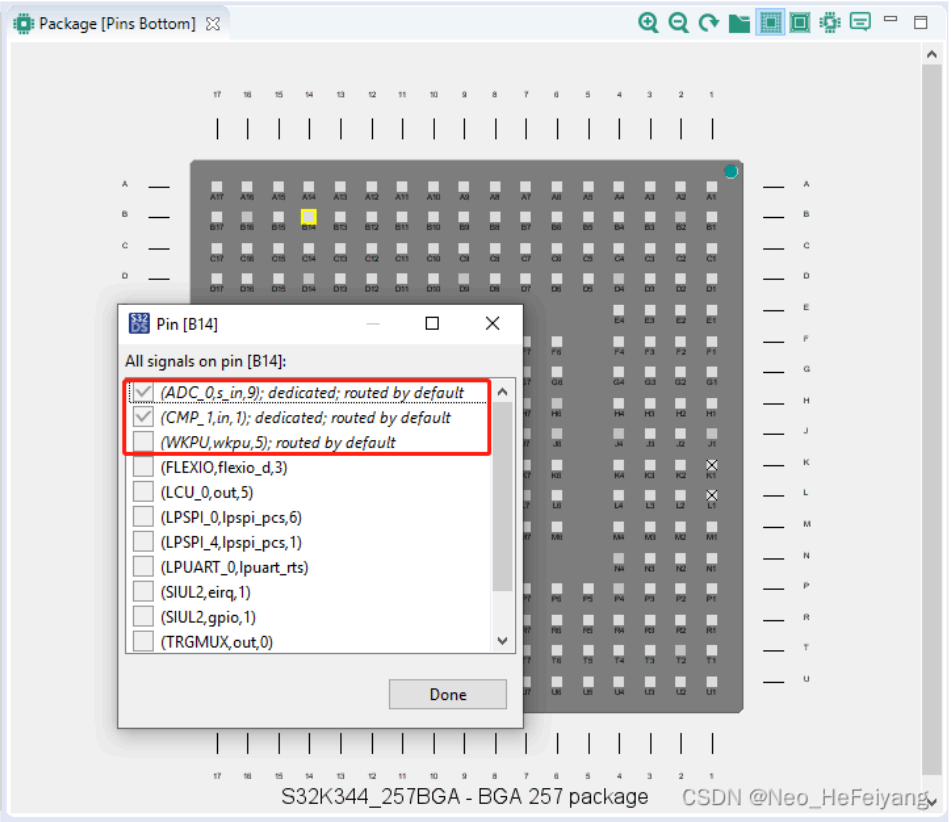
以PTA1为例

如图所示，可以看到ADC0_S9、CMP1_IN1、WKPU[5]功能选择寄存器是 - 的状态，即默认选通的。

| | | | | | | |
|---|---|---|---|---|---|---|
| PTA1 | SIUL_MSCR1 | 0000_0000 | GPIO[1] | SIUL | | I/O |
| | | 0000_0001 | LPSPI4_PCS1 | LPSPI4 | Peripheral Chip Select 1 | O |
| | | 0000_0010 | eMIOS_0_CH[9]_H | eMIOS_0 | eMIOS Channel | O |
| | | 0000_0011 | LPUART0_RTS | LPUART0 | Request To Send | O |
| | | 0000_0100 | FXIO_D3 | FXIO | FlexIO Bi-directional Shift/timer I/O | O |
| | | 0000_0101 | LCU0_OUT5 | LCU0 | LCU Output | O |
| | | 0000_0110 | LPSPI0_PCS6 | LPSPI0 | Peripheral Chip Select 6 | O |
| | | 0000_0111 | TRGMUX_OUT0 | TRGMUX | Trigger Mux Output | O |
| | - | - | ADC0_S9 | ADC0 | ADC Standard Input | I |
| | - | - | CMP1_IN1 | CMP1 | Comparator Input Signal | I |
| | - | - | WKPU[5] | WKPU | Wakeup Input | I |
| | SIUL_IMCR529 | 0000_0001 | EIRQ[1] | SIUL | External Interrupt | I |
| | SIUL_IMCR569 | 0000_0001 | eMIOS_0_CH[9]_H | eMIOS_0 | eMIOS Channel | I |
| | SIUL_IMCR667 | 0000_0001 | FXIO_D3 | FXIO | FlexIO Bi-directional Shift/timer I/O | I |
| | SIUL_IMCR739 | 0000_0001 | LPSPI0_PCS6 | LPSPI0 | Peripheral Chip Select 6 | I |
| | SIUL_IMCR768 | 0000_0001 | LPSPI4_PCS1 | LPSPI4 | Peripheral Chip Select 1 | I |

CSDN @Neo_HeFeiyang

同时通过S32Ds引脚配置也可以看出来，在这三个模块后面均有：routed by default.




相反的，哪些中断模块需要在IcuHwInterruptConfigList中添加就不言而喻了，比如我们常用的外部引脚中断就需要。

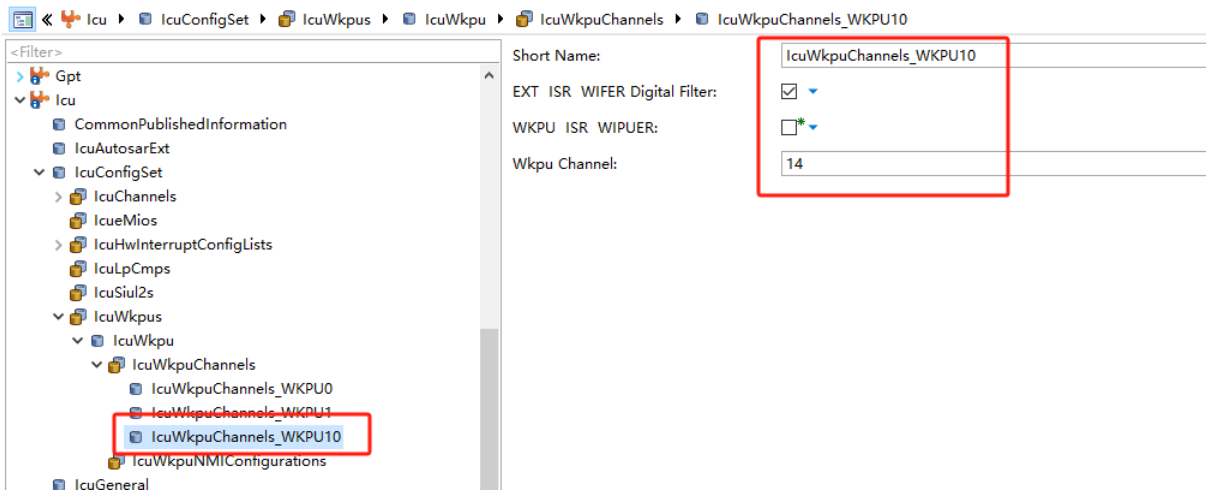
下面配PTC9 CAN1_RX 作为唤醒源：

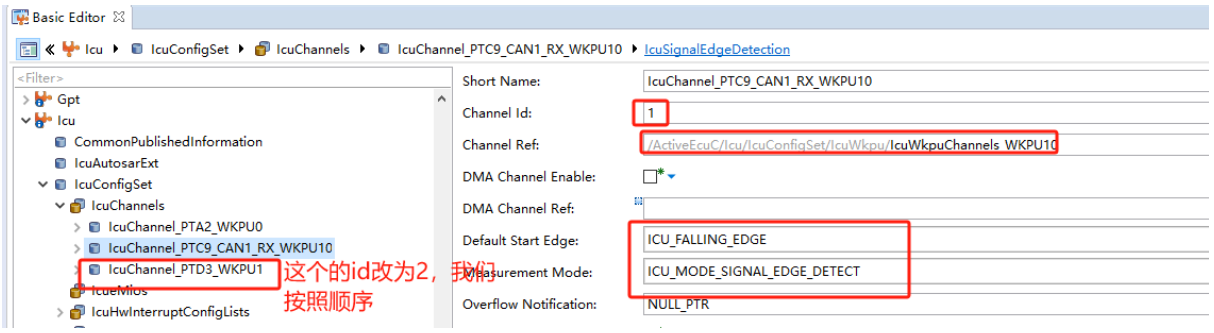
就是mcu休眠后，别的ECU节点可以通过发送报文来唤醒我们mcu，（因为现在我手上的板子CanTrcv没法配置为休眠，正常其他CanTrcv时有低功耗模式的，如可以通过CanTrcv的INH管脚唤醒MCU）,我们这里没有CanTrcv 中断唤醒MCU, 所以使用MCU 的CanRx 管脚来作为唤醒源，当外部有报文发送给CanTrcv ，我们CanRx管脚（WKPU10）就会有下降沿电平，进而唤醒MCU

port配置不变，还是复用选择IN  CAN1_RX的.。

需要注意的是，休眠前需要把CAN1_TX的通过port的接口改为输入GPIO，因为测试过程发现如果没有这个步骤，其他唤醒源唤醒时，这个CAN1_RX的WKPU10 会一直置位，可能原因时CANtx有发东西给CanTrcv, 因为CAN又是收发同时检测的，这个时候可能Rx也会有电平变化导致。

Icu




编译ICU模块

# 四、添加测试代码

休眠，这里测试先定义为 跑了10s 后调用休眠的代码

```
FUNC(void, CtApTestSwc_CODE) RCtApTestSwc_1000ms(void) /* PRQA S 0624, 3206 */ /* MD_Rte_0624, MD_Rte_3206 */
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>            DO NOT CHANGE THIS COMMENT!
 * Symbol: RCtApTestSwc_1000ms
 **********************************************************************************************************************/
  Dio_FlipChannel(DioConf_DioChannel_DioChannelPTA31_LED3_BLUE);
  //Uart_SyncSend(0, "Hello World\r\n",strlen("Hello World\r\n"),0xFFFF);
  static unsigned short u16Cnt = 0;
  printf("Hello World, %d\r\n",u16Cnt++);
  if(u16Cnt>=10){
    Mcu_InitClock(McuConf_McuClockSettingConfig_McuClockSettingConfig_Standby);
    Icu_EnableEdgeDetection(IcuConf_IcuChannel_IcuChannel_PTD3_WKPU1);
    Port_SetPinMode(PortConfigSet_PortContainerC_PortPinPTC8_CAN1_TX, PORT_GPIO_MODE);
    Port_SetPinDirection(PortConfigSet_PortContainerC_PortPinPTC8_CAN1_TX,PORT_PIN_IN);
    //Port_SetPinMode(PortConfigSet_PortContainerC_PortPinPTC9_CAN1_RX, PORT_GPIO_MODE);
    Icu_EnableEdgeDetection(IcuConf_IcuChannel_IcuChannel_PTC9_CAN1_RX_WKPU10);
    /* Enable pad Keeping in STABNBY IO configurations*/
    IP_DCM_GPR->DCMRWF1 &= ~(DCM_GPR_DCMRWF1_STANDBY_IO_CONFIG_MASK);
    Mcu_SetMode(McuConf_McuModeSettingConf_McuModeSettingConf_Standby);
    //Rte_Write_Request_ESH_RunRequest_0_requestedMode(RELEASED);
    //Rte_Write_Request_ESH_RunRequest_1_requestedMode(RELEASED);
  }
```

10s 后强制睡眠，使能WKPU1/10的检测，同时改变CAN1tx为输入

这句很重要，没有这局唤醒后所有唤醒标志都是1

**pad keeping配置**：用户在读取唤醒源的时候，**不需要考虑是fast唤醒还是Noramal唤醒**，也不需要用Standby RAM去存储唤醒源，因为MCU唤醒后，WKUP模块对应的寄存器都不会被复位，那么客户可以直接在main函数一开始先读取WISR_64 和WISR的值，然后把Pad_Keeping 功能Disable。

为什么用户读取的唤醒源和实际想要的唤醒源不一致，都是Pad_keeping的功能导致的

那么关于Pad_keeping功能的使用需要注意：

只要工程中有GPIO下降沿唤醒MCU，那么**必须使能Pad_keeping功能**。原因是如果睡眠之前没有使能Pad_Keeping，一旦MCU唤醒后，所有的GPIO都是高阻态，对应MCU GPIO内部电路就是产生了一个下降沿，所以会误导致WISR_64 或WISR对应唤醒置1；**通常情况都是建议**把Pad_Keeping 功能给使能，然后唤醒后，先读取WISR_64 和WISR的值，然后把Pad_Keeping 功能Disable

**注意**：

图中为fast exit，改为normal exit后退出状态不正常，原因是：MCU在进入低功耗之前把Standby IO enable了，需要手动在main函数引脚初始化前，加上下面一条语句

/* Disable standby IO pad keeping */

IP_DCM_GPR->DCMRWF1 |= DCM_GPR_DCMRWF1_STANDBY_IO_CONFIG_MASK;

当一个引脚在port和icu中被配置为wake up功能时，其同时在正常模式下具有中断的功能。要实现其正常模式下的中断功能，不需要额外的icu模块的配置，只需要在Platform中打开wake up中断功能即可。

唤醒，首先在EcuM的DriverInitZero的 usercode 添加读取wkpu的标志，必须要在前面读，否则后面DriverInitOne或者进入到swc_init时候读就没用了，mcu模块初始化会清掉的。

```
EcuM_AL_DriverInitZero()
 * ECUM_WKSOURCE_CN_CAN00_cda9acf5 (EcuM_WakeupSourceType)(32u)
 * ECUM_WKSOURCE_IcuWakeup_WKPU0   (EcuM_WakeupSourceType)(64u)
 */

/*****************************************************************************************************
 *  Additional configured User includes
 *****************************************************************************************************/
#include "BswM.h"
#include "Det.h"
#include "Rte_Main.h"
#include "OsIf.h"
#include "Mcu.h"
#include "Port.h"

/*****************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          <USERBLOCK User Includes>                    DO NOT CHANGE THIS COMMENT!
 *****************************************************************************************************/
#include "S32K344_WKPU.h"
/*****************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          </USERBLOCK>                                 DO NOT CHANGE THIS COMMENT!
 *****************************************************************************************************/
```

我们是K344的，这里是user的 define，davinci 生成ecum不会改这里

添加这个头文件是为了读取寄存器获取唤醒的flag

```
/*****************************************************************************************************
 *  CALLOUTS FROM THE STARTUP PHASE
 *****************************************************************************************************/


/*****************************************************************************************************
 * EcuM_AL_DriverInitZero
 *****************************************************************************************************/
FUNC(void, ECUM_CODE) EcuM_AL_DriverInitZero(void)
{
  BswM_InitMemory();
  Det_InitMemory();
  Rte_InitMemory();
  Det_Init( Det_Config_Ptr );

/*****************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          <USERBLOCK EcuM_AL_DriverInitZero>           DO NOT CHANGE THIS COMMENT!
 *****************************************************************************************************/
/* Add implementation of EcuM_AL_DriverInitZero  */
  extern uint32 u32WKPU_SR[2];
  /*Get and Store Wakeup events*/
  u32WKPU_SR[0] = IP_WKPU->WISR;
  u32WKPU_SR[1] = IP_WKPU->WISR_64;
  /*Clear Wakeup events*/
  IP_WKPU->WISR = u32WKPU_SR[0];
  IP_WKPU->WISR_64 = u32WKPU_SR[1];
  /* Disable standby IO pad keeping */
  IP_DCM_GPR->DCMRWF1 |= DCM_GPR_DCMRWF1_STANDBY_IO_CONFIG_MASK;
  return;
/*****************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          </USERBLOCK>                                 DO NOT CHANGE THIS COMMENT!
 *****************************************************************************************************/
}
```

读取唤醒事件的标志

回写，清除唤醒标志

disable pad保持功能

```
{
  Dio_FlipChannel(DioConf_DioChannel_DioChannelPTA29_LED1_RED);
}
uint32 u32WKPU_SR[2] = {0,0};
/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << End of documentation area >>                DO NOT CHANGE THIS COMMENT!
 *********************************************************************************************************/

FUNC(void, CtApTestSwc_CODE) RCtApTestSwc_Init(void) /* PROA S 0624, 3206 */ /* MD_Rte_0624, MD_Rte_3206 */
{
/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << Start of runnable implementation >>         DO NOT CHANGE THIS COMMENT!
 * Symbol: RCtApTestSwc_Init
 *********************************************************************************************************/
  printf("Swc Init, Reset Reason:%d\r\n",Mcu_GetResetReason());
  printf("Wakeup Src: WISR = 0x%08x, WISR_64= 0x%08x\r\n", u32WKPU_SR[0],u32WKPU_SR[1]);//IP_WKPU->WISR, IP_WKPU->WISR_64);
  Rte_Write_Request_ESH_RunRequest_0_requestedMode(REQUESTED);
  Rte_Write_Request_ESH_RunRequest_1_requestedMode(REQUESTED);
  Gpt_EnableNotification(GptConf_GptChannelConfiguration_GptChannelConfiguration_PIT0_CH0);
  Gpt_StartTimer(GptConf_GptChannelConfiguration_GptChannelConfiguration_PIT0_CH0, 40000000);

  Icu_EnableNotification(IcuConf_IcuChannel_IcuChannel_PTA2_WKPU0);
  Icu_EnableWakeup(IcuConf_IcuChannel_IcuChannel_PTA2_WKPU0);
  Icu_EnableEdgeDetection(IcuConf_IcuChannel_IcuChannel_PTA2_WKPU0);
/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << End of runnable implementation >>           DO NOT CHANGE THIS COMMENT!
 *********************************************************************************************************/
}


#define CtApTestSwc_STOP_SEC_CODE
#include "CtApTestSwc_MemMap.h" /* PROA S 5087 */ /* MD_MSR_MemMap */
```

打印唤醒原因
和唤醒flag

正常休眠后 ，我们把PTA2 接地， 这样就能出发WKPU0的下降沿唤醒

我们可以看到Mcu_GetResetReason()是 24 ，对应就是从wakeup唤醒，

WISR = 0x10， 那边就是对应WKPU0 是唤醒源

```
Swc Init, Reset Reason:24
Wakeup Src: WISR = 0x00000010, WISR_64= 0x00000000
Hello World, 0
ret=0, IsWrite=0, Read Data: 0x001b
Hello World, 1
Hello World, 2
Hello World, 3
Hello World, 4
Hello World, 5
ret=0, IsWrite=1, Write Data: 0x0001
Hello World, 6
Hello World, 7
Hello World, 8
```

CtApTestSwc.c * | EcuM_Callout_Stubs.c | Power_Ip_Cfg_Defines.h | CtApDiagSwc.c | CtApCanApp.c | CtApNvMSwc.c | ARMS

```
#define McuConf_McuResetReasonConf_MCU_DEBUG_FUNC_RESET    ((uint8)23U)

#define McuConf_McuResetReasonConf_MCU_WAKEUP_REASON    ((uint8)24U)

#define McuConf_McuResetReasonConf_MCU_NO_RESET_REASON    ((uint8)25U)
```

原因：

在60个外部引脚唤醒源之前还有四个特殊的唤醒源，因此0-59的外部引脚唤醒源对应的是4-63的唤醒通道。

This is the wake-up source mapping to WKPU:

- Wake-up source 0 : SWT_0 timeout, RTC-API API wake-up
- Wake-up source 1 : RTC-API RTC timeout
- Wake-up source 2 : Round robin wake-up interrupt (Trigger mode interrupt) from LPCMP_0, LPCMP_1, or LPCMP_2
- Wake-up source 3 : RTI wake-up
- Wake-up source 4 : Wake-up source 63—external pin wake-up sources, WKPU[0] - WKPU[59] CSDN @Neo_HeFeiyan

正常休眠后 ，我们把PTD3接地，WISR=0x20 ， WKPU1置位

```
Swc Init, Reset Reason:24[2025-08-29 02:43:01.322]
Wakeup Src: WISR = 0x00000020, WISR_64= 0x00000000[2025-08-29 02:43:01.329]
Hello World, 0[2025-08-29 02:43:01.329]
NmStateCan=5, NmModeCan=3[2025-08-29 02:43:01.515]
NmStateCan=5, NmModeCan=3[2025-08-29 02:43:01.713]
```

正常休眠后，我们通过上位机发送一条Can报文给板子，WISR=0x4000， WKPU10置位

```
Swc Init, Reset Reason:24[2025-08-29 02:44:11.485]
Wakeup Src: WISR = 0x00004000, WISR_64= 0x00000000[2025-08-29 02:44:11.499]
Hello World, 0[2025-08-29 02:44:11.596]
NmStateCan=5, NmModeCan=3[2025-08-29 02:44:11.782]
NmStateCan=5, NmModeCan=3[2025-08-29 02:44:11.974]
```